

Received April 28, 2019, accepted June 24, 2019, date of publication July 1, 2019, date of current version July 18, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2925828

An Optimization Method for Intrusion Detection Classification Model Based on Deep Belief Network

PENG WEI¹, YUFENG LI², ZHEN ZHANG¹, TAO HU¹, ZIYONG LI¹, AND DIYANG LIU¹

¹National Digital Switching System Engineering Technology Research Center, Zhengzhou 450002, China

²School of Computer Engineering and Science, Shanghai University, Shanghai 200444, China

Corresponding author: Yufeng Li (liyufeng_ndsc@163.com)

This work was supported in part by the Research and Development Program in Key Areas of Guangdong Province under Grant 2018B010113001, in part by the National Key Research and Development Program of China under Grant 2017YFB0803201, and in part by the National Natural Science Foundation of China for Innovative Research Groups under Grant 61521003.

ABSTRACT The rapid development and popularization of the network have brought many problems to network security. Intrusion detection technology is often used as an effective security technology to protect the network. The deep belief network (DBN), as a classic model of deep learning, has good classification performance and is often used in the field of intrusion detection. However, the network structure of DBN is generally set through practical experience. For the optimization problem of the DBN-based intrusion detection classification model (DBN-IDS), this paper proposes a new joint optimization algorithm to optimize the DBN's network structure. First, we design a particle swarm optimization (PSO) based on the adaptive inertia weight and learning factor. Second, we use the fish swarm behavior of cluster, foraging, and other behaviors to optimize the PSO to find the initial optimization solution. Then, based on the initial optimization solution, we use the genetic operators with self-adjusting crossover probability and mutation probability to optimize the PSO to search the global optimization solution. Finally, the global optimization solution constructed by the above-mentioned joint optimization algorithm is used as the network structure of the intrusion detection classification model. The experimental results show that compared with other DBN-IDS optimization algorithms, our algorithm shortens the average detection time by at least 24.69% on the premise of increasing the average training time by 6.9%; compared with the tested classification algorithms, our DBN-IDS improves the average classification accuracy by at least 1.3% and up to 14.80% in the five-category classification, which is proved to be an efficient DBN-IDS optimization method.

INDEX TERMS Intrusion detection, deep belief network, particle swarm optimization, artificial fish swarm algorithm, genetic algorithm.

I. INTRODUCTION

DBN is a classic model in deep learning. By learning a deep nonlinear network structure, the approximation of complex functions is realized. The purpose is to improve the accuracy of classification or prediction. Commonly used models in deep learning are mainly convolutional neural networks and automatic encoders, etc [1]. At the same time, DBN is also a deep neural network classifier composed of multiple Restricted Boltzmann Machines (RBMs) and a Back Propagation Neural Network (BPNN) [2]. It is widely used in speech recognition, image recognition and other fields.

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Quan.

The basic network structure, weight and threshold parameters of DBN are the key to the speed and accuracy of the classifier [3]. The basic network structure of the DBN refers to the DBN's depth and the number of its neurons per layer. Therefore, the performance of the DBN classifier can be optimized in two ways: by optimizing the basic network structure of DBN, that is, finding a suitable DBN's depth and the number of its neurons per layer. If the DBN network is too complicated, the DBN training time will be longer. If the network is too simple, it will affect the accuracy of DBN classification; by optimizing the weight or threshold parameters on the basic network structure in the DBN model. If the parameter settings are unreasonable, an effective training model cannot be generated, and the accuracy of the DBN

classification model can be easily reduced. In the case of uncertain DBN network structure, it is difficult to directly optimize the parameters of the DBN model, and the computational cost of directly optimizing the DBN network structure is less. Therefore, the DBN model is adjusted by optimizing the DBN network structure in our algorithm.

The DBN network structure optimization problem has fewer open documents. In fact, researchers always adopt empirical methods in the fields of image recognition and speech recognition to solve the problem of network structure [4]. At present, the methods that can be used for neural network structure or parameter optimization problems are mainly evolutionary computation [5], which is a swarm-oriented random search technique and method for simulating the biological evolution process in nature. However, researchers mainly use this method to the optimization of the neural network structure and parameters of the single hidden layer neural network, lacking of related research on optimization multi-hidden layer neural network classification model.

The DBN model is often used as the intrusion detection classification model. In order to optimize the multi-hidden layer neural network in the DBN model, we propose an artificial fish swarm algorithm optimization PSO (AFSA-PSO) joint genetic algorithm optimization PSO (GA-PSO) algorithm. It is used to construct and optimize the DBN-IDS model, mainly to optimize the basic network structure of DBN. It is worth mentioning that the weight and threshold parameters of this paper are mainly generated automatically based on the basic network structure of DBN according to the DBN adaptive adjustment mechanism [6]. Our contributions of this paper are summarized as follows.

- We propose a DBN-IDS model optimization method for optimizing the basic network structure of the DBN classification model with multiple hidden layers. Firstly, the AFSA is used to optimize the PSO algorithm to obtain the initial optimal particles. Then, the initial optimal particles are used as the initial particle swarm of the GA optimization PSO algorithm. In the case of ensuring the classification accuracy, the global optimal particle is finally optimized and used as the basic network structure of the DBN classification model.
- By contrast, we study the performance of DBN classification model optimized by our method in 2-category classification and multiclass classification. Moreover, we study the performance of other DBN model optimization algorithms, and some machine learning methods such as Support Vector Machine (SVM), Random Forest and Naive Bayes in 2-category classification and multiclass classification based on the benchmark NSL-KDD dataset.
- We compare the performance of DBN classification model optimized by our method with other DBN model optimization algorithms and some machine learning methods both in 2-category classification and multiclass classification. The experimental results illustrate that our method is very suitable for intrusion detection,

the performance of our method is superior to other DBN model optimization algorithms and machine learning methods on the NSL-KDD dataset in both binary and multiclass classification. It improves the accuracy of intrusion detection, providing a new research method for intrusion detection.

The rest of the paper is organized as follows: In Section II, we review the related research in the field of intrusion detection, focusing on the application of deep learning methods in intrusion detection and the application of intelligent algorithms in optimizing intrusion detection classification models based on neural network. In Section III, we propose a construction and optimization method for DBN-IDS. In Section IV, by comparing and discussing the experimental results of this algorithm with other classification algorithms, the superiority of our method is demonstrated. Finally, the conclusions are discussed in Section V.

II. RELATED WORK

In prior studies, many machine learning methods are used in intrusion detection, where machine learning methods are mainly divided into unsupervised learning, semi-supervised learning and supervised learning. Supervised learning methods, such as Support Vector Machine (SVM) and KNN algorithms, unsupervised learning methods, such as cluster algorithms, have been widely used in intrusion detection. Related references have given comparative analysis of supervised learning and unsupervised learning techniques in the application of intrusion detection. Literature [7] validates the classification performance comparison of NSL-KDD datasets in different machine learning methods, such as decision trees, SVM, and Bayesian.

In recent years, deep learning, a branch of machine learning, has become increasingly popular and has been applied for intrusion detection; studies have shown that deep learning completely surpasses traditional methods. There are also many neural network classification models applied to intrusion detection, such as Deep Neural Networks (DNN), DBN, etc, which have advantages in processing and learning the temporal or spatial characteristics of data. In a neural network, the hidden layer neural nodes are summed from the weights of the upper nodes, and the results are transmitted to the next layer by nonlinear transformation. The last layer of the network represents the representation learning of the input data. Literature [8] applies DBN classifier to the intrusion detection model. Based on the KDD CUP99 dataset, it is found that DBN's performance is better than SVM and artificial neural network (ANN). Literature [9] proposes intrusion detection model based on DBN and probabilistic neural network (PNN). The DBN model is used to reduce the intrusion data dimension, and PNN is used to classify the reduced data. The experimental results show that DBN-PNN classification performance is better than traditional PNN. Literature [10] proposes an intrusion detection optimization model based on deep stacking autoencoder (SAE). Literature [11] applies the semi-supervised learning method based on random weighted

neural networks (NNRW) to the IDS architecture, and proves the good performance of the method through the NSLKDD data set.

To achieve the required approximation accuracy, various learning algorithms for RBFNNs have been proposed to adjust the model parameters (ie, the coordinates of RBF centers, the widths of RBF neurons, and the output weights) and to determine the network size (the number of RBF neurons). Literature [12] optimizes the weight matrix and threshold of BP neural network (BPNN) as a classifier through particle swarm optimization (PSO). Literature [13] uses RBF neurons as a classifier, the radio basis function neural networks (RBFNN) optimizes the width of the RBFNN hidden layer and the network weight between the hidden layer and the output layer through PSO. In [3], a self-organizing radial basis function (SORBF) neural network is used as a classifier. The nonlinear regression function is used to adjust the inertia weight of PSO, and the number of single hidden layer nodes and network weights of SORBF is improved by using improved PSO. The traditional PSO-based network structure optimization algorithm is optimized for the parameters of single hidden layer network and the number of hidden layer nodes in BP, RBF, etc., lacking the researches of optimizing the structure of multi-hidden layer neural network, and the traditional optimization algorithm of network structure is simple and lacking pertinence.

In our algorithm, the DBN model is used as the intrusion detection classification model. In order to optimize the multi-hidden layer neural network in the DBN model, we propose AFSA optimization PSO joint GA optimization PSO algorithm (AFSA-GA-PSO). It is used to build and optimize the DBN-IDS model. Its main task is to find the basic network structure of DBN. Compared with previous work, we design a new joint optimization algorithm to effectively solve the optimization problem of multi-layer hidden layer neural network, and evaluate them in network intrusion by using NSL-KDD's training data set and test data set. The performance of the two-category and multi-classification is tested, and they are compared with the DBN classification model constructed by the traditional optimization algorithm, and compared with the machine learning model with superior performance such as SVM, Random Forest and Naive Bayes.

III. PROPOSED METHODOLOGIES

This paper proposes a DBN-IDS model construction and optimization method, which is mainly used to optimize the basic network structure of DBN. The number of input layers of the DBN network structure is related to the number of data features, and the number of output layers is related to the number of categories. Therefore, it is only necessary to find the hidden layer structure, corresponding to the number of hidden layer nodes in each layer and each dimension in the particle, the integer particle swarm generated by the random method is used as the candidate multiple sets of initial network structures. We need to define the relevant equation, such as particle swarm $swarm =$

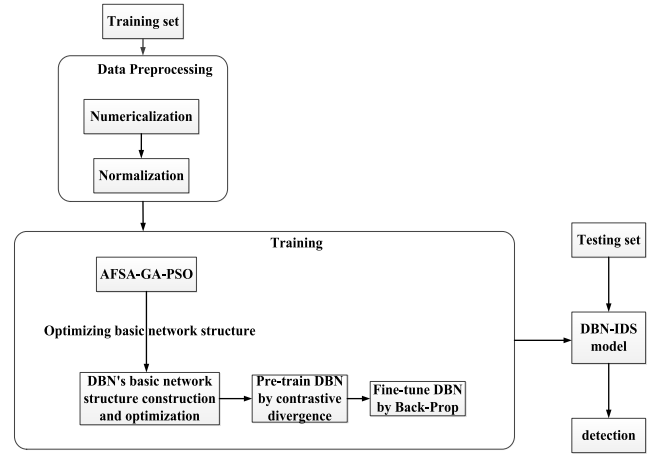


FIGURE 1. The summary process of intrusion detection classification model construction and optimization method.

$[x_1, x_2, \dots, x_i, \dots, x_n]^T$, particle swarm's velocity $vstep = [vstep_1, vstep_2, \dots, vstep_i, \dots, vstep_n]^T$, and i^{th} particle position $x_i = [num(h_{i1}), num(h_{i2}), \dots, num(h_{id}), \dots, num(h_{in})]$. the speed of the i^{th} particle is $vstep_i$, and we can also make $vstep_i = [vs_{i1}, vs_{i2}, \dots, vs_{id}, \dots, vs_{in}]$, $smin \leq num(h_{id}) \leq smax$, $vmin \leq vs_{id} \leq vmax$, where $smin$ and $smax$ indicates the minimum and maximum values of the elements in the particle position, $vmin$ and $vmax$ indicates the minimum and maximum values of the elements in the particle velocity, both $num(h_{id})$ and vs_{id} are integers.

An overall framework of DBN-IDS model is trained in four stages, as shown in Fig. 1. Firstly, the training subset is preprocessed, and the content of subset is digitized and normalized [14], referring to Section IV for details; secondly, the training subset after preprocessing is used to train the DBN-IDS model. The AFSA-GA-PSO method is used to optimize the best fitness particles, which can be used to construct the network structure of the intrusion detection classification model, referring to Section III; thirdly, the pre-training procedure consisting of learning a stack of RBMs with the greedy contrastive divergence algorithm, after the pre-training, a DBN is constructed by unrolling the RBMs and then the DBN is fine-tuned using back-propagation of error derivatives [15]; lastly, the DBN network structure constructed and optimized by our method is used as the basic network structure of the intrusion detection classification model, and the pre-processed test subset is used to detect the attack behavior and the performance analysis of the classification model.

For the fitness standard of particle swarm in AFSA-GA-PSO part of Fig. 1, the fitness function proposed is as shown in formula (1).

$$\begin{aligned}
 Fitness = & (1 - a - b - c - d - e) * ERRate \\
 & + a * \frac{n}{\maxdim} + b * \frac{\sum(x_i)}{n * smax} \\
 & + c * FPRate + d * FNRate + e * DRate \quad (1)
 \end{aligned}$$

where $a, b, c, d, e \in [0, 1]$ indicate the weight parameter. $ERRate$ denotes the error rate, as in formula (2); n denotes

TABLE 1. Network attack type prediction matrix.

	Real type	Normal data	Abnormal data
Forecast type			
Normal data		TN	FN
Abnormal data		FP	TP

the number of DBN hidden layers; $maxdim$ denotes the maximum number of DBN hidden layers; $smax$ denotes the maximum number of neurons in the hidden layer; $sum(x_i)$ denotes the total number of all hidden layer nodes in DBN, and $sum(x_i) = \sum_{d=1}^n num(h_{id})$; $FPRate$ denotes the false positive rate, $FNRate$ denotes the false negative rate, and $DRate$ denotes the detection rate. The update is as shown in formula (4), (5), and (6) [16]. Among them, *Accuracy*, *FPRate*, *FNRate*, and *DRate* can be used as evaluation indicators for optimization algorithm performance in the intrusion detection classification model; the lower the fitness, the better the optimized network structure.

$$ERRate = 1 - Accuracy \quad (2)$$

where *Accuracy* denotes the classification accuracy rate, as in formula (3).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

$$FNRate = \frac{FN}{TN + FN} \quad (4)$$

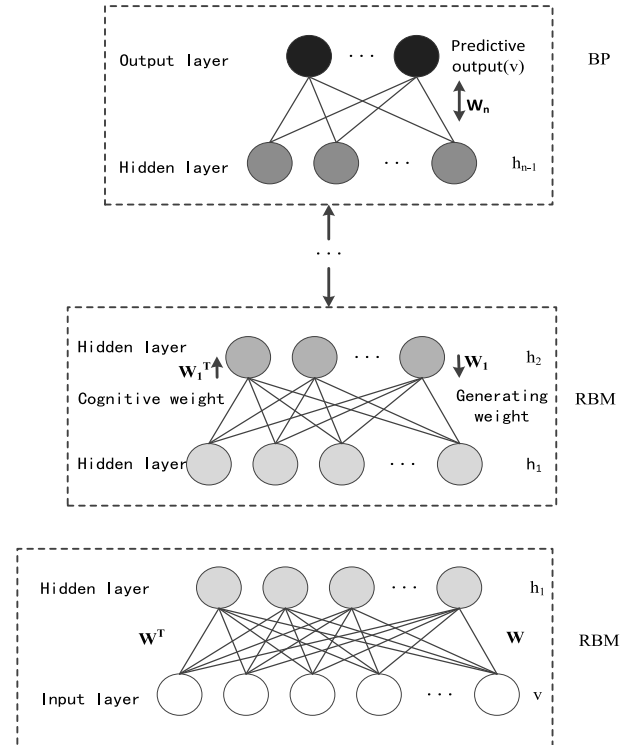
$$FPRate = \frac{FP}{TP + FP} \quad (5)$$

$$DRate = \frac{FN}{TP + FN} \quad (6)$$

In the intrusion detection, where TP and TN denote the normal data and abnormal data correctly classified respectively; FP and FN denote the normal data and abnormal data of the misclassification respectively, that is, false positives and false negatives, which is shown in Table 1.

By constructing the fitness function as in formula (1), under the premise of ensuring accuracy, this method can find the optimal configuration combination of DBN hidden layer depth dim and overall network structure x_i , and realize the optimization of DBN network structure.

As shown in Fig. 2, the DBN consists of multiple RBM and a BP neural network. Wherein, the RBM is a two-layer neural network consisting of an input layer and an implicit layer. The RBM lacks a direct connection between the nodes of the same layer. There is only a connection between the hidden layer and the hidden layer. The BP neural network is a multi-layer feedforward neural network trained by error back propagation, which is mainly used to classify data. The DBN training process extracts and abstracts the input data of the input layer from the bottom up using each layer of RBM, and outputs the information through the hidden layer as the input data of the upper layer network. The upper layer network refers to the RBM network or the top BP neural network, and the

**FIGURE 2.** DBN network structure.

number of RBMs is n ($n > 1$) which is equal to the number of hidden layers of the DBN, and the d^{th} layer of hidden layer is expressed as h_d ($d = 1, 2, \dots, n$), the number of its nodes is expressed as $num(h_d)$.

The DBN basic network structure of this paper includes the input layer, the output layer and the number of neuron nodes in each hidden layer. The number of input layer nodes is related to the number of features, and the number of output layer nodes corresponds to the total number of data categories. Therefore, it is only necessary to find the number of hidden layers and $num(h_d)$ ($d = 1, 2, \dots, n$). For this reason, evolutionary computing method such as PSO, AFSA, etc, can be introduced to optimize the network structure.

A. PARTICLE SWARM OPTIMIZATION

Particle Swarm Optimization (PSO) is a swarm intelligence algorithm designed by predation behavior of birds. The basic thought is to search for optimal solutions through the cooperation and information sharing between particles in particle swarm. PSO can be divided into binary PSO and continuous PSO. Continuous PSO refers to the assignment of real numbers to particles, and binary PSO assigns binary numbers to particles. Because continuous PSO has broader application prospects, this paper uses continuous PSO for feature selection, and optimizes according to individual particle extremum and swarm optimal solution. The optimization formulas for speed and position are formulas (7) and (8), respectively:

$$vstep_{id}^{t+1} = w_i \cdot vstep_{id}^t + c_1 \cdot rand_{i1} \cdot (pbest_{id}^t - x_{id}^t) + c_2 \cdot rand_{i2} \cdot (gbest_d^t - x_{id}^t) \quad (7)$$

$$x_{id}^{t+1} = x_{id}^t + vstep_{id}^{t+1} \quad (8)$$

where t is the number of iterations; $i = 1, 2, \dots, m$ (m denotes the number of all particles); $d = 1, 2, \dots, n$ (n denotes the total number of dimensions of the particle); $rand_{i1}$ and $rand_{i2}$ are random number between 0 and 1; $pbest_{id}^t$ denotes the d^{th} dimensional optimal position of the historical extremum of i^{th} particle in the t^{th} iteration; $gbest_d^t$ denotes the d^{th} dimensional optimal position of the swarm optimal particle in the t^{th} iteration; c_1 and c_2 denote the learning factor; x_{id}^t denotes the d^{th} position of i^{th} particle in the t^{th} iteration; $vstep_{id}^t$ denotes the d^{th} dimensional moving speed of i^{th} particle in the t^{th} iteration.

Literature [17] studied four kinds of PSO inertia weights w , such as linear, concave and convex function reduction, and found that in multi-peak function optimization, convex function reduction inertia weight method has the fastest convergence rate and the best effect. Therefore, the inertia weight of this paper takes the value as shown in formula (9).

$$w_i = (w_{\max} - w_{\min}) \cdot (1 - \frac{iter}{maxiter})^3 + w_{\min} \quad (9)$$

where $w_{\max}$ and $w_{\min}$ denote the maximum and minimum values of the inertia weight respectively; $iter$ denotes the number of iterations; $maxiter$ denotes the maximum number of iterations.

c_1 embodies the ability of particles to learn from themselves, and c_2 embodies the ability of particles to learn from swarm. The decrease of c_1 and the increase of c_2 are beneficial to the initial exploration of particles and the cognitive ability of the swarm in the later stage. Therefore, the learning factors c_1 and c_2 are assigned as in formulas (10) and (11) respectively.

$$c_1 = c_3 - c_4 \cdot (\frac{iter}{maxiter}) \quad (10)$$

$$c_2 = c_4 + c_3 \cdot (\frac{iter}{maxiter}) \quad (11)$$

where c_3 and c_4 are constants, and $c_3 > c_4$.

B. AFSA OPTIMIZATION PSO ALGORITHM

The artificial fish swarm algorithm is an optimization strategy based on animal autonomy [18]. The basic thought is that the food with the most fish accumulation is also very rich. Artificial fish are constructed according to the above characteristics, which simulate fish swarm's cluster, rear-end and foraging behavior [19], through the virtual waters of the fish swarm to achieve optimization.

In this paper, the fish swarm thought is used to optimize the PSO to find the initial optimal particles. The cluster, rear-end and foraging behaviors in the AFSA are used to improve the moving mode of the PSO algorithm. The basic flow of the particle swarm algorithm based on the fish swarm thought is as **Algorithm 1**.

The value of the element in x_i^t , x_i^{t+1} are integers and range from $smin$ to $smax$. The fitness is solved by the formula (1).

Algorithm 1 AFSA Optimization PSO Algorithm

Input: Field of view: Visual

Moving step: Step

Number of trials of foraging behavior: *trytimes*

Maximum fitness: *maxfit*

Number of iterations: *iter* = 0

The maximum number of iterations of AFSA-PSO: *fmaxiter*

Output: Initial optimization solution set *temppbest*

Step1: Parameter initialization

1. Randomly initialize the particle swarm *swarm* according to the n value.

2. Solve the *swarm* fitness *fswarm*.

3. Initialize *pbest* to *swarm*.

4. Initialize *gbest* to the optimal fitness particle in the *swarm*.

5. Solve the *gbest* fitness *fgbest*.

6. Initialize the extremum record matrix *temppbest_i* of the i^{th} particle *pbest* and its fitness matrix *ftemppbest_i*.

Step2: End condition judgment

7. **if** the current iteration number *iter* ≤ *fmaxiter* or *fgbest* ≤ *maxfit*

Jump to **Step3**.

else

Jump to **Step7**.

endif

Step3: Particles perform cluster behavior

8. Detect the neighborhood particle center x_j .

9. **if** *Fitness_i* > *Fitness_j*

Go to the center position x_j as in formula (12), and skip to **Step2**.

else

Skip to **Step4**.

endif

Step4: Particle perform foraging behavior

10. **while** the number of trials is not satisfied with *trytimes*, **do**

A particle x_k is randomly selected within its Visual, and x_k is as in formula (13).

if *Fitness_i* > *Fitness_k*

Go further in this direction, as in formula (14), and skip to **Step2**.

else

Re-randomly select the state x_k .

if x_k satisfies the forward condition

break;

endif

end while

Step5: Particles perform random behavior

11. Randomly selects new particle in the field of view.

12. Moves toward the particle as a default behavior, as in formula (15).

Step6: Update and record particle swarm related properties

Algorithm 1 (Continued.) AFSA Optimization PSO Algorithm

13. Update particle inertia *weight*, the learning factors c_1 and c_2 as in formulas (9), (10), and (11), particle swarm's *velocity* and *position*, $pbest$, $gbest$, $fpbest$, and $fgbest$.

14. Record each particle's $pbest$ to $temppbest$, let $iter = iter + 1$, and jump to **Step2**.

Step7: Get global optimization solution

15. Use $temppbest$ as a initial optimization solution set

The PSO is optimized by the behavior of cluster and foraging of fish swarm. The cluster behavior makes the particles closer to the center of the neighboring particles with better fitness and no crowding [20]. It is beneficial to increase the particle swarm movement mode and overcome the problem that the particle swarm is easy to fall into the local optimal solution. The original movement behavior of the particle swarm is similar to the rear-end behavior of the fish swarm. Since the historical extreme value of the single particle is recorded and the implementation is simple, cluster behavior can find a better optimization solution than the rear-end behavior.

Although the AFSA optimization PSO (AFSA-PSO) can find the initial optimization particles, it is difficult to achieve further optimization. Based on the optimization of initial optimization particles, this paper introduces GA optimization PSO (GA-PSO) algorithm and find global optimization particles.

$$x_i^{t+1} = x_i^t + (x_j - x_i^t) * rand \quad (12)$$

where function *random* generates a number ranging from 0 to 1.

$$x_k = x_i^t + 2 \cdot (rand - 0.5) \cdot Visual \quad (13)$$

where Visual denotes the fish's field of vision.

$$x_i^{t+1} = x_i^t + 2 \cdot (rand - 0.5) \cdot Step \cdot (x_k - x_i^t) \quad (14)$$

where Step denotes the step size of the fish.

$$x_i^{t+1} = x_i^t + 2 \cdot (rand - 0.5) \cdot Step \quad (15)$$

C. GA OPTIMIZATION PSO ALGORITHM

Genetic algorithm is a randomized search method that finds the optimal solution by simulating the natural evolution process. The crossover operator and mutation operator in the Genetic algorithm have great effect on PSO algorithm, which can increase the diversity of particle swarm change, producing a swarm representing the new solution set, and overcome the problem that particle swarm is easy to fall into local optimum [21].

In our method, in order to avoid the problem of premature convergence of particle swarm, the particle swarm algorithm crosses multiple pairs of particles at each iteration, and uses the roulette algorithm based on fitness value to select multiple pairs of parent particles and cross them [22]. Produce multiple

pairs of next-generation particles and select the parent particles and historical extremum in the superior next-generation particle replacement swarm.

The crossover operation mainly exchanges the first parent particle dimensions between 1 and $\lfloor L/2 \rfloor$ with the second parent particle dimensions between $L - \lfloor L/2 \rfloor + 1$ and L , and L denotes the total particle dimension. The number of cross-operating particles per iteration is as shown in formula (16).

$$P_i = \lfloor swarmsize - (\frac{iter}{maxiter})^2 \cdot (swarmsize - 2) \rfloor \quad (16)$$

where swarmsize denotes the number of all particles.

According to the characteristics of particle swarm, the particle swarm types are more random and diverse at first, and the particle swarm gradually converges in the later stage, and the particle swarm similarity increases. We can easily know that the cross-operation is more effective than the later stage. Therefore, as the number of iterations increases, the number of cross-operation particles per iteration should decrease, and the crossover probability should decrease accordingly, which helps to reduce the computational cost. The crossover probability is updated as in formula (17).

$$pcross = (pc_{max} - pc_{min}) \cdot (1 - \frac{iter}{maxiter}) + pc_{min} \quad (17)$$

where pc_{min} and pc_{max} denote the minimum and maximum crossover probabilities, respectively.

Although the search space can be well explored at the beginning of the crossover operation, it has a small effect after the particle swarm is converged in the late stage [19]. The mutation operation and the cross operation are just the opposite. In the initial stage of the search, the effect is not obvious, but in the later stage, the search effect is better. Therefore, the mutation operation is introduced to further improve the spatial search ability, and the mutation probability increases as the number of iterations increases. This will help to further find better particles. The mutation probability is updated as in the formula (18); the selection probability of the d^{th} position in the particle is as shown in formula (19).

$$pmutation = (pm_{max} - pm_{min}) \cdot \frac{iter}{maxiter} + pm_{min} \quad (18)$$

$$CR_d = \begin{cases} \frac{(gbest_d - \theta)^2}{1 - \theta} & IF \theta < gbest_d \leq 1 \\ \frac{(\theta - gbest_d)^2}{\theta} & IF 0 \leq gbest_d \leq \theta \end{cases} \quad (19)$$

where pm_{min} and pm_{max} denote the minimum and maximum mutation probabilities; $gbest_d$ denotes the global optimal particle d^{th} dimensional position; θ denotes the selection threshold. When the value of the d^{th} dimensional position of $gbest$ is larger than the threshold θ , if the value of $gbest_d$ is larger, the CR_d is larger, otherwise the CR_d is smaller. When the value of the d^{th} dimensional position of $gbest$ is smaller than the threshold θ , if the value of $gbest_d$ is smaller, the CR_d is larger, otherwise the CR_d is smaller. Then generate a random number r ranging from 0 to 1. If $r > CR_d$, perform the corresponding mutation operation, such as formula (20),

otherwise the new d^{th} dimensional position of the new particle is assigned to $gbest_d$.

$$child_d = \begin{cases} \theta + (1 - \theta) \cdot \frac{gbest_d}{\theta} & \text{IF } 0 \leq gbest_d \leq \theta \\ \theta - \frac{(1 - gbest_d)}{1 - \theta} \cdot \theta & \text{IF } \theta < gbest_d \leq 1 \end{cases} \quad (20)$$

wherein, if the value of $gbest_d$ is less than or equal to the selection threshold θ , the value of $child_d$ is mapped to a real number ranging from θ to 1, otherwise the value of $child_d$ is mapped to a real number ranging from 0 to θ , which is beneficial to change the selection properties of the feature.

Therefore, the basic flow of our GA-PSO algorithm is shown in **Algorithm 2**. It can find a better global optimization solution based on the initial solution obtained by **Algorithm 1**.

D. AFSA-GA-PSO ALGORITHM APPLIED TO OPTIMIZE THE DBN-IDS MODEL

In summary, this paper proposes a hybrid particle swarm optimization algorithm based on AFSA and GA (AFSA-GA-PSO) applied to optimize DBN-IDS model. The algorithm firstly optimizes the particle by using the AFSA optimization PSO algorithm, and then uses the GA optimization PSO to find the global optimization solution for the initial optimal particle search. The particle swarm size is n_2 , and the number of non-zero elements in the particle is consistent with the number of hidden layers. In our algorithm, particle representation network structure has $length$ hidden layers, and i^{th} particle is $x_i = [num(h_{i1}), num(h_{i2}), \dots, num(h_{ilength})]$. Particle velocity is $vstep_i = [vs_{i1}, vs_{i2}, \dots, vs_{ilength}]$, and its length depends on the dimensions of the initial optimal particle. The basic flow of the algorithm is as follows:

In **Algorithm 3**, the purpose of performing random out-of-order operation on $pbest$ is to randomly disturb the order of all particle historical extremum positions to find better individual particle extremum, which is beneficial to avoid the particle individual extremum falling too fast into the local optimal solution.

The AFSA optimization PSO algorithm mimics the flight process of the swarm of birds, and uses rear-end, foraging and cluster behaviors in the AFSA to enrich and optimize the flight behavior of the swarm, which is beneficial to finding the initial optimization solution with better effect. The initial optimization solution is used as the initial solution of GA optimization PSO algorithm. The genetic operators in the GA optimization PSO algorithm can accelerate the process of the particle swarm through a reasonable genetic search mechanism.

IV. EXPERIMENT RESULTS AND DISCUSSION

In this research, the experiment is performed on a personal computer Lenovo Tianyi 510 which has a configuration of an Intel(R) Core(TM) i5-8400 CPU @2.80 GHz, 8GB memory. This chapter is divided into four parts. First, **Part A** of this chapter introduces the NSL-KDD dataset used in the experiment. Second, **Part B** introduces the data

Algorithm 2 GA Optimization PSO Algorithm

Input: Particle swarm: $swarm_2$

Maximum fitness: $maxfit$

Number of iterations: $iter_2 = 0$

Minimum and maximum values of crossover probability: $pcmin, pcmax$

Minimum and maximum values of mutation probability: $pmmin, pmmax$

The maximum number of iterations of GA-PSO: $gmaxiter$

Output: Global optimization extreme value $gbest$

Step1: Parameter initialization

1. The characteristics of the intrusion detection data set are assigned to different real numbers ranging from 0 to 1, which constitute the particle position vector.
2. Randomly initialize the particle swarm *position* and *velocity*, solve the fitness.
3. Initialize the historical extremum $pbest$ of particle in $swarm_2$, and the $swarm_2$'s optimal solve $gbest$ and their fitness $fswarm_2, fpbest, fgbest$.

Step2: End condition judgment

4. **if** $iter > gmaxiter$ or $fgbest > maxfit$ is satisfied

Execute **Step6**.

else

Execute **Step3**.

endif

Step3: The specific implementation steps

5. The inertia weight w and the learning factors c_1 and c_2 are updated as in formulas (9), (10), and (11), and the crossover and mutation probabilities are updated as in formulas (17) and (18), respectively.
6. Perform cross-operations on particle pairs in $swarm_2$.
7. Perform random out-of-order operations on $pbest$.
8. Perform mutation operations on $gbest$ to obtain respective descendant particles.
9. Replace the parent particles with the better descendant particles.

Step4: Update the extreme values of particles

10. Update the corresponding $pbest$, $gbest$ and their fitness separately.

Step5: Update particle speed and position

11. Update the speed and position of the particle swarm, let $iter_2 = iter_2 + 1$, and then skip to **Step2**.

Step6: Get global optimization solution

12. Use $gbest$ as a global optimization solution

preprocessing process. Third, **Part C** introduces the experimental parameter settings. Last, **Part D** designs the experiments and discusses the results.

The method of Section III is used to construct and optimize the intrusion detection classification model. By optimizing the basic network structure of DBN, the DBN classification model is constructed and applied to IDS, and good experimental results are obtained. Two experiments have

Algorithm 3 AFSA-GA-PSO Algorithm

Input: Maximum particle velocity: v_{max}
 Minimum velocity: v_{min}
 The maximum width of DBN's hidden layer: s_{max}
 The minimum width of DBN's hidden layer: s_{min}
 The initial hidden layer number of DBN: $n = 1$
 The maximum hidden layer number of DBN: n_{max}
Output: Global optimization extreme value g_{best}
Step1: Set the number of hidden layers of the DBN
 1. if $n \leq n_{max}$
 Skip to **Step2**.
 else
 Skip to **Step7**.
 endif
Step2: Execute AFSA Optimization PSO algorithm
 2. Particle swarm $swarm$ searches initial optimization solution set $temppbest$ referring to **Algorithm 1**
Step3: Solve the initial particle swarm
 3. Solve the fitness of particle swarm's $temppbest$.
 4. Assign the first n pbest of the $temppbest$ to the new particle $swarm_2$ and update the $swarm_2$'s fitness.
Step4: Execute GA Optimization PSO algorithm
 5. Particle swarm $swarm_2$ continues to search global optimization solution referring to **Algorithm 2**.
Step5: Record global optimization solution
 6. The global optimized particle g_{best} obtained after each iteration is regarded as the optimized network structure with the hidden layer number of DBN as n , record it, and let $n = n + 1$, then jump to **Step1**.
Step6: Select the optimal network structure
 7. The DBN hidden layer structure with the best fitness in the DBN hidden layer number n ($n = 1, 2, \dots, n_{max}$) is selected.
Step7: Construct an optimized DBN-IDS model
 8. The final DBN-IDS model is constructed based on the hidden layer structure of DBN with best fitness.

been designed to study the performance of the 2-category classification (normal, abnormal) and multiply classification in DBN-IDS model. In order to compare with other DBN model optimization algorithms and machine learning methods, a comparative experiment is designed. In the 2-category classification experiment, we compare our algorithm with four DBN-IDS model optimization algorithm, such as CMPSO [23], PSO [24], our branching algorithms—AFSA optimization PSO (AFSA-PSO) and GA optimization PSO (GA-PSO), and machine learning method, such as Naive Bayes, Random Forest, SVM.

In the same way, we analyze the multi-classification of the DBN-IDS model based on the NSL-KDD dataset. By contrast, we study the performance of five DBN-IDS model optimization algorithm, Naive Bayes, Random Forest, and Support Vector Machine methods in the

TABLE 2. Different classifications in the NSL-KDD dataset.

Data set type Attack type	KDDTrain+	KDDTest+	KDDTest-21
Normal	65120	11536	1987
Probe	10786	2213	2351
Dos	36944	6251	3051
U2R	52	200	200
R2L	992	2653	2653
Total	113894	22853	10242

TABLE 3. Features of NSL-KDD dataset.

Feature classification	Feature name
Basic characteristics of TCP connections (9)	(1)duration,(2)protocol_type,(3)service,(4)flag,(5)src_bytes,(6)dst_bytes,(7)land,(8)wrong_fragment,(9)urgent
Content characteristics of TCP connections (13)	(10)hot,(11)num_failed_logins,(12)logged_in,(13)num_compromised,(14)root_shell,(15)num_root,(16)su_attempted,(17)num_file_creations,(18)num_shells,(19)num_access_files,(20)num_outbound_cmds,(21)is_host_login,(22)is_guest_login
Time-based network traffic statistics (9)	(23)count,(24)srv_count,(25)error_rate,(26)srv_error_rate,(27)reror_rate,(28)srv_reror_rate,(29)same_srv_rate,(30)diff_srv_rate,(31)srv_diff_host_rate
Host-based network traffic statistics (10)	(32)dst_host_count,(33)dst_host_srv_count,(34)dst_host_same_srv_rate,(35)dst_host_diff_srv_rate,(36)dst_host_same_src_port_rate,(37)dst_host_srv_diff_host_rate,(38)dst_host_error_rate,(39)dst_host_srv_error_rate,(40)dst_host_reror_rate,(41)dst_host_srv_reror_rate

5-category classification. Finally, we compare the performance of the DBN-IDS model with traditional methods.

A. DATASET DESCRIPTION

The NSL-KDD data set is based on the standard data set of the intrusion detection domain—KDD CUP99 data set [25]. It solves the problems of redundant features and duplicate records of the traditional KDD CUP99 data set. It has been widely used in the field of network intrusion detection. Each network connection in the NSL-KDD data set is marked as normal and abnormal. The exception types are divided into Probe (scan and probe), DoS (denial of service attack), U2R (illegal access to local superuser), and R2L (unauthorized remote access), including a total of 39 types of attacks. The NSL-KDD sample data set selected is shown in Table 2, and its data characteristics are described in Table 3.

In Table 2, KDDTrain+ is the training data set, KDDTest+ and KDDTest-21 are test data sets, and KDDTest-21 is a subset of KDDTest+, which is more difficult to classify. In Table 3, feature dimension (2), (3), (4), (7), (12), (21), (22)

are discrete features, and the rest are continuous features. The test set lacks some types of attacks in the training set, which helps to better simulate the real environment of intrusion detection.

B. DATA PREPROCESSING

Floating point numbers must be entered in the input layer neurons of the deep belief network. The floating point values range from 0 to 1. There are two stages to preprocessing the data, as follows:

1) NUMERICALIZATION

Using the new code mapping method, the protocol_type feature can be mapped to an ordered number, such as protocol type: tcp = [1, 0, 0], protocol type: udp = [0, 1, 0], protocol type: icmp3 = [0, 0, 1]. Likewise, a symbol feature “service” with 53 different values and a “flag” feature with 10 different values can be mapped to an ordered number. Finally, 41 features are digitized into 104 features.

2) NORMALIZATION

In order to eliminate the dimensional impact of each feature, the training set and test set must be normalized. According to the following data transformation formula (21), each value in the data set obtained in the first stage is normalized over the ranging 0 to 1.

$$Y = \frac{Y_{original} - Y_{min}}{Y_{max} - Y_{min}} \quad (21)$$

where $Y_{original}$ is the original value of the y feature, Y_{min} is the minimum value of the y feature, and Y_{max} is the maximum value of the y feature.

C. PARAMETER SETTINGS

The five optimization algorithm common parameters for optimizing the basic network structure are shown in Table 4. The non-common parameters of our algorithm are set as follows: weight $w_{max} = 0.9$, $w_{min} = 0.4$; learning factor $c_1 = 1.5$, $c_2 = 1.0$; number of trials of cluster behavior $Trytimes = 5$; field of view Visual = 150; moving step Step = 5; the number of iterations $f_{maxiter} = 60$, $g_{maxiter} = 40$; wherein, setting the number of iterations $f_{maxiter}$ slightly more than $g_{maxiter}$ is more conducive to optimizing the initial solution, and does not affect the search for global optimization solutions. Moreover, we set the number of iterations for the other four optimization algorithms to 100.

D. EXPERIMENT RESULTS AND DISCUSSION

1) BINARY CLASSIFICATION

In our experiment, we have mapped 41-dimensional features into 104-dimensional features, thus the DBN-IDS model has 104 input nodes, and 2 output nodes in the binary (2-category) classification experiments. The number of epochs is given 2 for the KDDTrain+ dataset.

The DBN's basic network structure optimized by the five optimization algorithms in the experiment is shown

TABLE 4. Five optimization algorithms common parameter settings.

Common parameter	Assignment
Particle maximum and minimum velocity are v_{max} and v_{min} , respectively.	$v_{max}=100$, $v_{min}=-100$
The maximum number of RBM pre-training is $RBMnumepochs$	$RBMnumepochs=2$
The maximum number of BP fine-tuning is $BPnumepochs$	$BPnumepochs=8$
The maximum width of the hidden layer is s_{max}	$s_{max}=300$
The minimum width of the hidden layer is s_{min}	$s_{min}=5$
Weight parameter of fitness formula (1): a, b, c, d, e	$a=0.002, b=0.028, c=0.02, d=0.02, e=0.025, (1-a-b-c-d-e)=0.90$
Minimum fitness is $minfitness$	$minfitness=0.01$
The maximum number of DBN hidden layers is n_{max}	$n_{max}=10$
Particle swarm size n_2	$n_2=10$
RBM pre-training learning rate $rate1$	$rate1=0.01$
BP fine-tuning learning rate $rate2$	$rate2=0.1$
Input layer node number is $inputnode$	$inputnode=104$
Output layer node number is $outputnode$	$outputnode=2$ or 5
The maximum DBN hidden layer number is n_{max}	$n_{max}=10$

TABLE 5. Optimal network structure of DBN model obtained by five optimization algorithms.

Classifier	Basic network structure of DBN-IDS
AFSA-GA-PSO-DBN	[75 33 18 12]
PSO-DBN	[167 105 58 28 13 8]
GA-PSO-DBN	[95 45 25 12 6]
AFSA-PSO-DBN	[148 86 38 15]
CMPSO-DBN	[92 45 23 13 5]

TABLE 6. Confusion matrix of 2-category classification on KDDTest+.

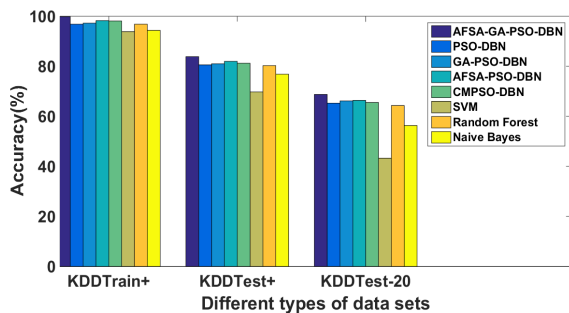
	Real type	Normal data	Abnormal data
Forecast type			
Normal data		9763	1915
Abnormal data		1773	9402

in Table 5. From the table, we find that compared with other optimization algorithms, the optimal network structure of the DBN model obtained by AFSA-GA-PSO and its branch AFSA-PSO algorithm has only four layers, and the total number of hidden layer nodes is the least, which is beneficial to improve detection speed in intrusion detection.

The confusion matrix of our algorithm applied to DBN-IDS on the testing set KDDTest+ in the 2-category classification experiments is shown in Table 6. As can be seen from Table 6, TN is 9763, TP is 9402, FN is 1915, and FP is 1773 which means that our algorithm works with classification accuracy (83.86%) for the KDDTest+ dataset.

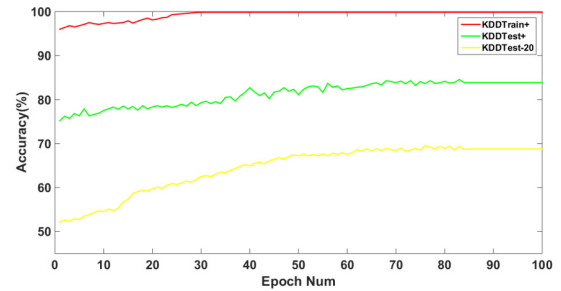
TABLE 7. Average classification accuracy of all the classification algorithms in 2-category classification.

Classifier	Classification accuracy under different data sets (%)		
	KDDTrain+	KDDTest+	KDDTest-20
AFSA-GA-PSO-DBN	99.85	83.86	68.75
PSO-DBN	96.83	80.58	65.27
GA-PSO-DBN	97.26	81.02	66.17
AFSA-PSO-DBN	98.27	81.98	66.37
CMP-PSO-DBN	98.12	81.23	65.53
SVM	93.87	69.75	43.28
Random Forest	96.83	80.27	64.32
Naive Bayes	94.37	76.83	56.32

**FIGURE 3.** Comparison of average classification accuracy of 2-category classification algorithms.

Literature [24] has shown the results obtained by Naive Bayes, Random Forest, and Support Vector Machine. Table 7 shows the comparison of the average classification accuracy of DBN-IDS model optimized by optimization algorithms and other classification algorithms. After calculation, compared with other classification algorithms, it is found that our algorithm reduces the average classification accuracy by at least 1.95% and the average classification accuracy by up to 15.18%.

We can find that compared with other classification algorithms, the average classification accuracy of the algorithm is significantly higher than other algorithms as shown in Fig. 3. The DBN model network structure obtained by our algorithm is superior to other optimization algorithms and has lower fitness value, which has higher average classification accuracy. Wherein, the classification accuracy rate obtained by AFSA-PSO algorithm is at a high level in the optimization algorithm; the classification accuracy rate obtained by GA-PSO algorithm is at a medium level. By combining the above two algorithms to obtain our algorithm, it is found in the experimental test that the classification accuracy of DBN-IDS can be effectively improved. This shows that in our algorithm, AFSA-PSO algorithm itself has superior performance, but it has certain limitations, cannot find better optimization solution; GA-PSO algorithm performance is general. However, in the case that the pre-algorithm obtains

**FIGURE 4.** The Accuracy on the KDDTrain+, KDDTest+ and KDDTest-21 datasets in the 2-category Classification.**TABLE 8.** Optimal network structure of DBN model obtained by five optimization algorithms.

Classifier	Basic network structure of DBN-IDS
AFSA-GA-PSO-DBN	[81 42 26 13]
PSO-DBN	[132 101 55 35 21 10]
GA-PSO-DBN	[118 69 38 25 12]
AFSA-PSO-DBN	[72 35 18 9]
CMP-PSO-DBN	[93 42 29 18 7]

a good basic solution, GA-PSO algorithm can find a better global optimization solution.

After 100 iterations, the classification accuracy of our algorithm is stable at 68.75% for the KDDTest-21 dataset, 83.86% for the KDDTest+ dataset, and 99.85% for the KDDTrain+ dataset as shown in Fig. 4.

2) MULTICLASS CLASSIFICATION

In the 5-category classification experiments, we have mapped 41-dimensional features into 104-dimensional features, thus the DBN-IDS model has 104 input nodes, and 5 output nodes. Table 8 shows the optimal network structure of DBN optimized by the five optimization algorithm in the experiment. The experimental results are similar to the 2-category classification; only differ in the number of hidden layer nodes per layer.

In order to compare the performance of different DBN-IDS model optimization algorithms on the benchmark dataset for the multiclass classification experiments, Five DBN-IDS optimization algorithms are used to train models through the training set (using 10-layer cross-validation) [26]. We then apply the models to the testing set. The results are described in Table 9. It shows the average training time and the average detection time of the DBN-IDS optimized by our algorithm. After calculation, it is found that compared with other DBN-IDS model optimization algorithm, our algorithm only increases the average training time by 6.9%, but shortens the average detection time by 24.69%.

We can find that in different data sets, although the training time of our algorithm is higher than other algorithms by the lower percentage, while the detection time of our algorithm is higher than other algorithms by the higher percentage as shown in Fig. 5 and Fig. 6. This shows that although the time complexity of our algorithm has increased slightly,

TABLE 9. Average training time and average detection time obtained by five optimization algorithms.

Classifier	Data set	KDDTest+	KDDTest-20
AFSA-GA-PSO-DBN	Average training time /s	13358.32	8201.29
	Average detection time /s	1813.96	1076.93
PSO-DBN	Average training time /s	11325.35	7428.74
	Average detection time /s	2265.26	1369.34
GA-PSO-DBN	Average training time /s	12335.45	7789.93
	Average detection time /s	2573.92	1872.97
AFSA-PSO-DBN	Average training time /s	15321.43	8575.45
	Average detection time /s	2142.14	1287.26
CMPSO-DBN	Average training time /s	10353.01	7528.49
	Average detection time /s	2321.62	1521.73

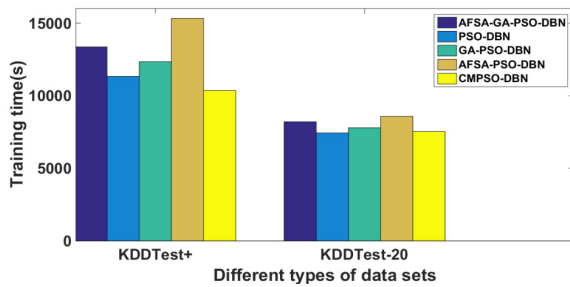


FIGURE 5. Comparison of average training time of five optimization algorithms.

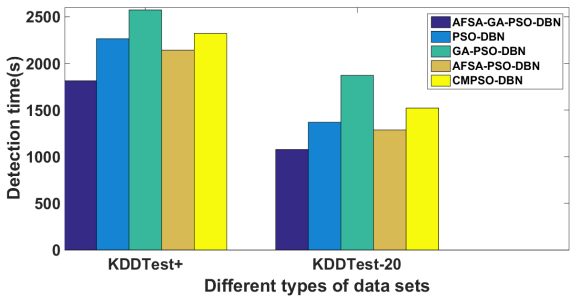


FIGURE 6. Comparison of average detection time of five optimization algorithms.

it can effectively find a better DBN-IDS model. Wherein, AFSA-PSO algorithm has the longest training time, but the detection time is only longer than our algorithm; the training time and detection time obtained by GA optimization PSO are at medium level in the optimization algorithm. Our algorithm combines the above two algorithms, although at the cost of increasing the training time, it is beneficial to obtain the optimal DBN-IDS that makes the detection time shorter.

In order to compare the performance of different classification algorithms, based on the above experiments, we apply classic machine learning algorithm, such as Naive Bayes, Random Forest, and Support Vector Machine to train classification algorithms through the training set. We then apply the models to the testing set. The results are described in Table 10. It shows the average accuracy of the classification algorithms. Compared with the 2-category classification, the accuracy

TABLE 10. Average classification accuracy of all the classification algorithms in 5-category classification.

Classifier	Classification accuracy under different data sets (%)		
	KDDTrain+	KDDTest+	KDDTest-20
AFSA-GA-PSO-DBN	98.55	82.36	66.25
PSO-DBN	95.72	79.22	63.21
GA-PSO-DBN	96.35	80.42	63.27
AFSA-PSO-DBN	97.26	80.68	65.31
CMPSO-DBN	97.11	80.13	63.43
SVM	92.27	68.25	42.25
Random Forest	95.36	78.87	63.12
Naive Bayes	93.27	75.43	55.48

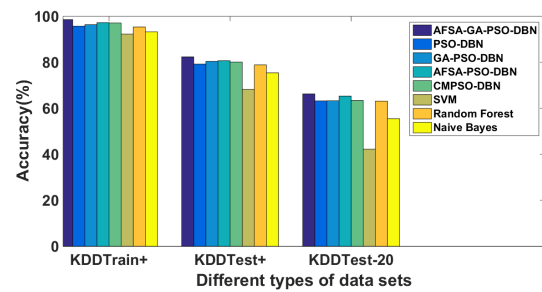


FIGURE 7. Comparison of average classification accuracy of all the classification algorithms.

TABLE 11. 5-category classification data prediction matrix based on KDDTest+.

Forecast type \ Real type	Normal	Probe	DoS	U2R	R2L
Normal	9221	175	202	18	175
Probe	603	1849	234	0	134
Dos	812	37	5451	14	114
U2R	303	17	173	168	97
R2L	897	135	191	0	2133

of classification algorithms is declined in the 5-category classification. After calculation, it is found that compared with other classification algorithms, our algorithm reduces the average classification accuracy by at least 1.30% and the average classification accuracy by up to 14.80%.

We can find that the experimental results are similar to the 2-category classification as shown in Fig. 7. The only difference is that the overall classification accuracy of classification algorithm in the 5-category experiment is lower than the 2-category classification experiment.

The confusion matrix of the AFSA-GA-PSO applied to DBN-IDS model on the test set KDDTest+ in the 5-category classification experiments as shown in Table 11. The experiment shows that the accuracy of our model is 82.36% for KDDTest+ and 66.25% for KDDTest-21, which is better than those obtained using Naive Bayes, Random Forest, Support

TABLE 12. Multi-class evaluation matrix.

Attack type	Probe	DoS	U2R	R2L
DR(%)	83.55	87.20	84.00	80.40
FPR(%)	0.77	5.64	0.17	3.02

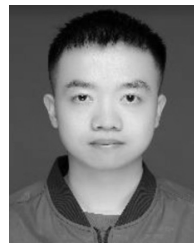
Vector Machine and the other classification model. The detection rate and false positive rate of the different attack types is shown in Table 12.

V. CONCLUSION

DBN is a deep learning model widely used in speech recognition, image recognition and other fields. Its good classification performance makes it suitable for intrusion detection. This paper proposes a DBN-IDS model construction and optimization method. The PSO algorithm based on fish swarm thought is used to find the initial optimization solution. The initial optimization solution is used as the initial particle swarm of GA optimization PSO algorithm. Finally, the DBN-IDS model is optimized by our algorithm. The experimental results show that the DBN-IDS model optimized by our algorithm has higher detection speed and detection accuracy. However, the proposed method is to optimize the DBN network structure in the range of limited hidden layers. If the maximum number of hidden layers in the DBN is set too large, the training time will have a greater impact on the fitness, which is not conducive to find a well-functioning DBN-IDS model. The next step is to explore a more appropriate fitness function, throw away the excessive impact of training time on fitness, based on the DBN's network structure optimized by our algorithm, the weights or threshold parameters on the DBN network structure are optimized to further optimize the DBN-IDS model.

REFERENCES

- [1] M. Fatahi, M. Shahsavari, M. Ahmadi, A. Ahmadi, P. Boulet, and P. Devienne, "Rate-coded DBN: An online strategy for spike-based deep belief networks," *Biologically Inspired Cogn. Archit.*, vol. 24, pp. 59–69, Apr. 2014.
- [2] L. Yu, R. Zhou, T. Ling, and R. Chen, "A DBN-based resampling SVM ensemble learning paradigm for credit classification with imbalanced data," *Appl. Soft Comput.*, vol. 69, pp. 192–202, Aug. 2018.
- [3] H. G. Han, W. Lu, Y. Hou, and J. F. Qiao, "An adaptive-PSO-based self-organizing RBF neural network," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 104–117, Jan. 2018.
- [4] F. C. Chen and M. R. Jahanshahi, "NB-CNN: Deep learning-based crack detection using convolutional neural network and Naïve Bayes data fusion," *IEEE Trans. Ind. Electron.*, vol. 65, no. 5, pp. 4392–4400, May 2018.
- [5] J. Tian, M. Li, F. Chen, and N. Feng, "Learning subspace-based RBFNN using coevolutionary algorithm for complex classification tasks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 1, pp. 47–61, Jan. 2016.
- [6] D. Papamartzivanos, F. G. Mármol, and G. Kambourakis, "Introducing deep learning self-adaptive misuse network intrusion detection systems," *IEEE Access*, vol. 7, pp. 13546–13560, 2019.
- [7] D. K. Bhattacharyya and J. K. Kalita, *Network Anomaly Detection: A Machine Learning Perspective*, vol. 45. Boca Raton, FL, USA: CRC Press, 2013, pp. 455–463.
- [8] N. Gao, L. Gao, Q. Gao, and H. Wang, "An intrusion detection model based on deep belief networks," in *Proc. 2nd Int. Conf. Adv. Cloud Big Data*, Nov. 2014, pp. 247–252.
- [9] G. Zhao, C. Zhang, and L. Zheng, "Intrusion detection using deep belief network and probabilistic neural network," in *Proc. IEEE Int. Conf. Comput. Sci. Eng.*, Jul. 2017, pp. 639–642.
- [10] Y. W. Zhan, "The applications of deep learning on traffic identification," in *Proc. Blackhat*, 2015, pp. 1411–1420.
- [11] R. A. R. Ashfaq, X.-Z. Wang, J. Z. Huang, H. Abbas, and Y.-L. He, "Fuzziness based semi-supervised learning approach for intrusion detection system," *Inf. Sci.*, vol. 378, pp. 484–497, Feb. 2017.
- [12] P. Q. Bao and F. M. Yang, "Intrusion detection based on KCPA and SVM," *Comput. Appl. Softw.*, vol. 2, no. 23, pp. 125–127, 2006.
- [13] Z. Lin, G. Chen, W. Guo, and Y. Liu, "PSO-BPNN-based prediction of network security situation," in *Proc. 3rd Int. Conf. Innov. Comput. Inf. Control*, Jun. 2008, p. 37.
- [14] M. Al-Qatf, Y. Lasheng, M. Al-Habib, and K. Al-Sabahi, "Deep learning approach combining sparse autoencoder with SVM for network intrusion detection," *IEEE Access*, no. 6, pp. 52843–52856, 2018.
- [15] Z. W. Wei and Y. Bin, "Research on optimal strategy of multi-stage intrusion detection game in WSNs," *J. Electron. Inf. Technol.*, vol. 40, no. 1, pp. 63–71, 2018.
- [16] W. Wang, Y. Sheng, J. Wang, X. Zeng, X. Ye, Y. Huang, and M. Zhu, "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2018.
- [17] Y. Liu, X. F. Tian, and Z. H. Zhan, "Research on inertia weight control method based on particle swarm optimization algorithm," *J. Nanjing Univ. Natural Sci. Ed.*, vol. 47, no. 4, pp. 364–371, 2011.
- [18] G. Zhou, Y. Li, Y.-X. He, X. Wang, and M. Yu, "Artificial fish swarm based power allocation algorithm for MIMO-OFDM relay underwater acoustic communication," *IET Commun.*, vol. 12, no. 9, pp. 1079–1085, May 2018.
- [19] L. W. Zhan, C. Z. Yi, K. X. Ying, Y. Y. Hai, X. An, and L. You, "Approach to WTA in air combat using IAFA-SA-IHS algorithm," *J. Syst. Eng. Electron.*, vol. 29, no. 3, pp. 519–529, Jun. 2018.
- [20] Z. Qiang, W. Q. Hui, and L. W. Hong, "VNE-AFS: Network virtualization mapping algorithm based on artificial fish stocks," *Trans. Commun.*, vol. 1, pp. 170–177, Aug. 2012.
- [21] Y. Song, F. Wang, and X. Chen, "An improved genetic algorithm for numerical function optimization," *Appl. Intell.*, vol. 49, no. 5, pp. 1880–1902, Dec. 2018.
- [22] D. Gong, J. Sun, and Z. Miao, "A set-based genetic algorithm for interval many-objective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 22, no. 1, pp. 47–60, Feb. 2018.
- [23] H. B. Nguyen, X. Bing, P. Andreae, and M. Zhang, "Particle Swarm Optimisation with genetic operators for feature selection," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, Jun. 2017, pp. 286–293.
- [24] H.-M. Feng, "Self-generation RBFNs using evolutionary PSO learning," *Neurocomputing*, vol. 70, nos. 1–3, pp. 241–251, Dec. 2006.
- [25] Bay and Hettich. (2018). *The UCI KDD Archive*, Department of Information and Computer Science. [Online]. Available: <http://kdd.ics.uci.edu>
- [26] (2016). *Weka 3-Data Mining With Open Source Machine Learning Software in Java*. [Online]. Available: <http://www.cs.waikato.ac.nz/ml/weka/>



PENG WEI was born in 1994. He is currently pursuing the master's degree with the National Digital Switching System Engineering and Technology Research Center, Zhengzhou, China. His research interest includes new-generation information communication networks.



YUFENG LI was born in 1976. He is currently a Professor with the School of Computer Engineering and Science, Shanghai University. His current research interests include broadband information networks and high-speed router core technology.



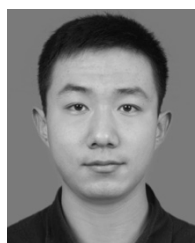
ZHEN ZHANG was born in 1985. He is currently a Lecturer with the National Digital Switching System Engineering Technology Research Center. His current research interests include network measurement and network management.



ZIYONG LI was born in 1995. He received the B.E. degree from the Harbin Institute of Technology, in 2017. He is currently pursuing the master's degree with the National Digital Switching System Engineering and Technology Research Center, Zhengzhou, China. His research interests include new-generation information communication networks, software-defined networking, and network security.



TAO HU received the B.E. degree from Xi'an Jiaotong University. He is currently pursuing the Ph.D. degree in network cyber security with the National Digital Switching System Engineering and Technological Research Center, Zhengzhou, China. His research interests include software-defined networking, control plane, and network security.



DIYANG LIU was born in 1995. He received the B.E. degree from the Beijing Institute of Technology, in 2018. He is currently pursuing the master's degree with the National Digital Switching System Engineering and Technology Research Center, Zhengzhou, China. His research interests include robustness of complex networks, software-defined networking, and network security.

...